

9

Vid 1 (Introduction)

10

- foundation of neural networks } course 1
 * deep learning.

11

- Improving Neural networks: Hyperparameter tuning

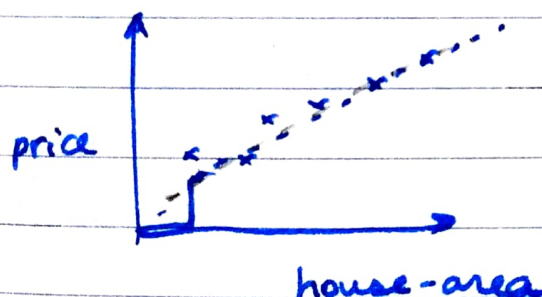
12

Vid 2.

1 pm

price vs house-area

2

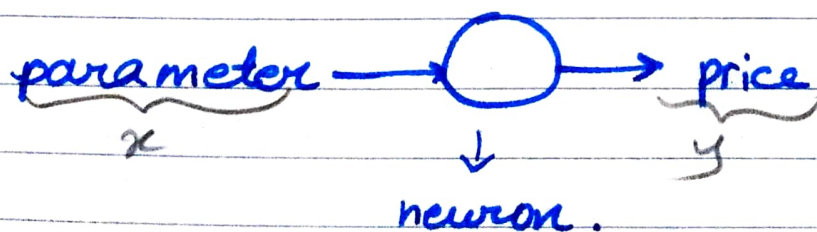


3

you can predict future house prices based on the parameter 'house-area'.

4

5



6

neuron calculates a ~~new~~ linear function to predict house prices.

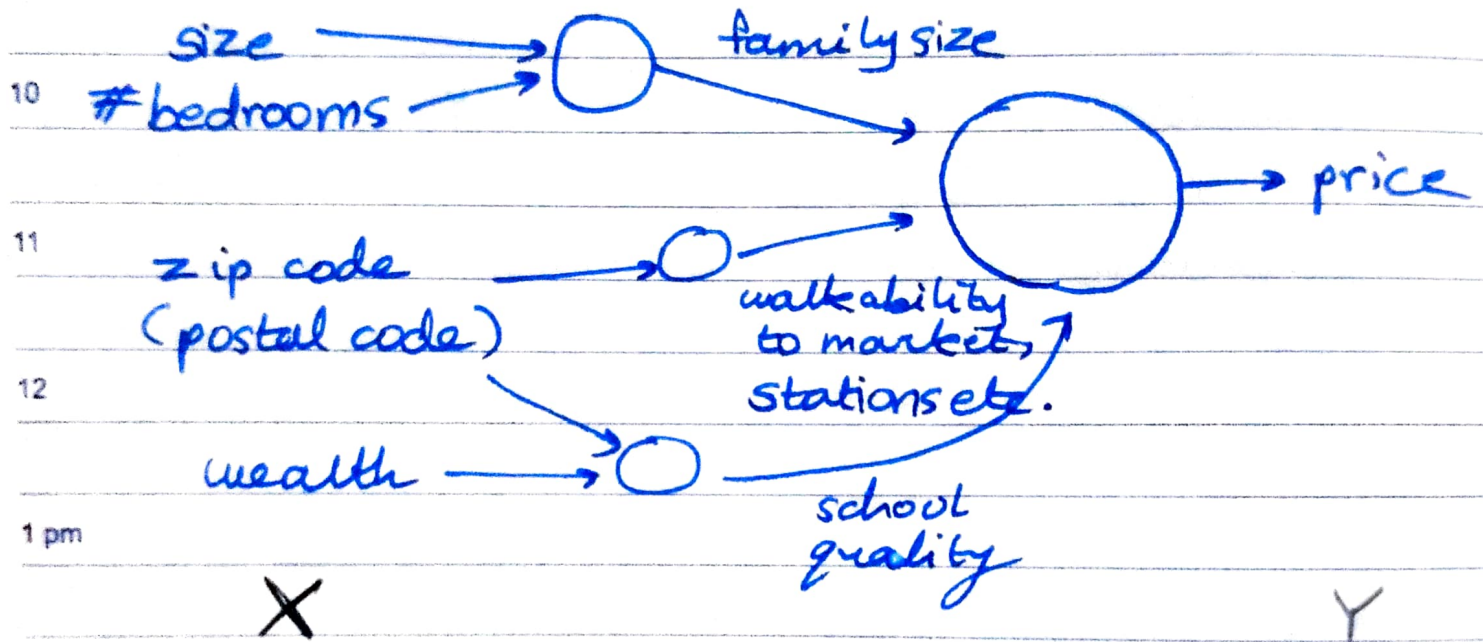
7

In this example, function is called "Rectified Linear Unit".



A larger neural network is made of smaller neurons.

Housing Price more variables:



If you give enough (x, y) pairs to neural network, it improves its ability to generate y accurately for a given x .

Vid 3.

"Supervised Learning"

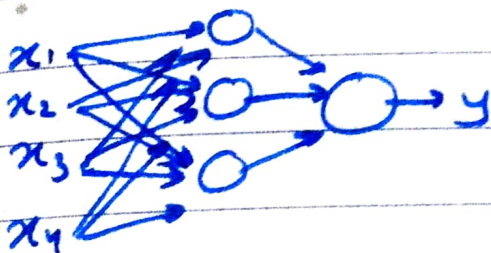
we want to learn a function that maps
if x to an o/p y .

8 am

example of NN

9

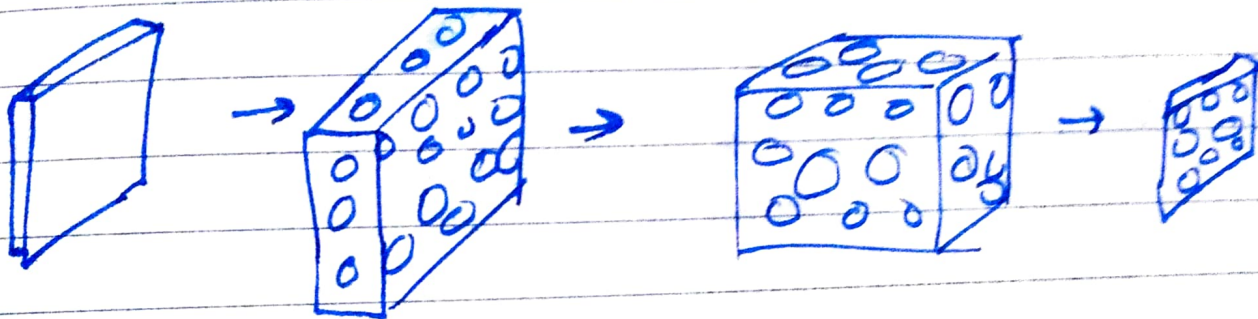
10



"STANDARD NN"

11

12



1 pm

"CONVOLUTIONAL NN"

2

Structured data

unstructured data.

3

parameters

O/p

audio

4

image



5

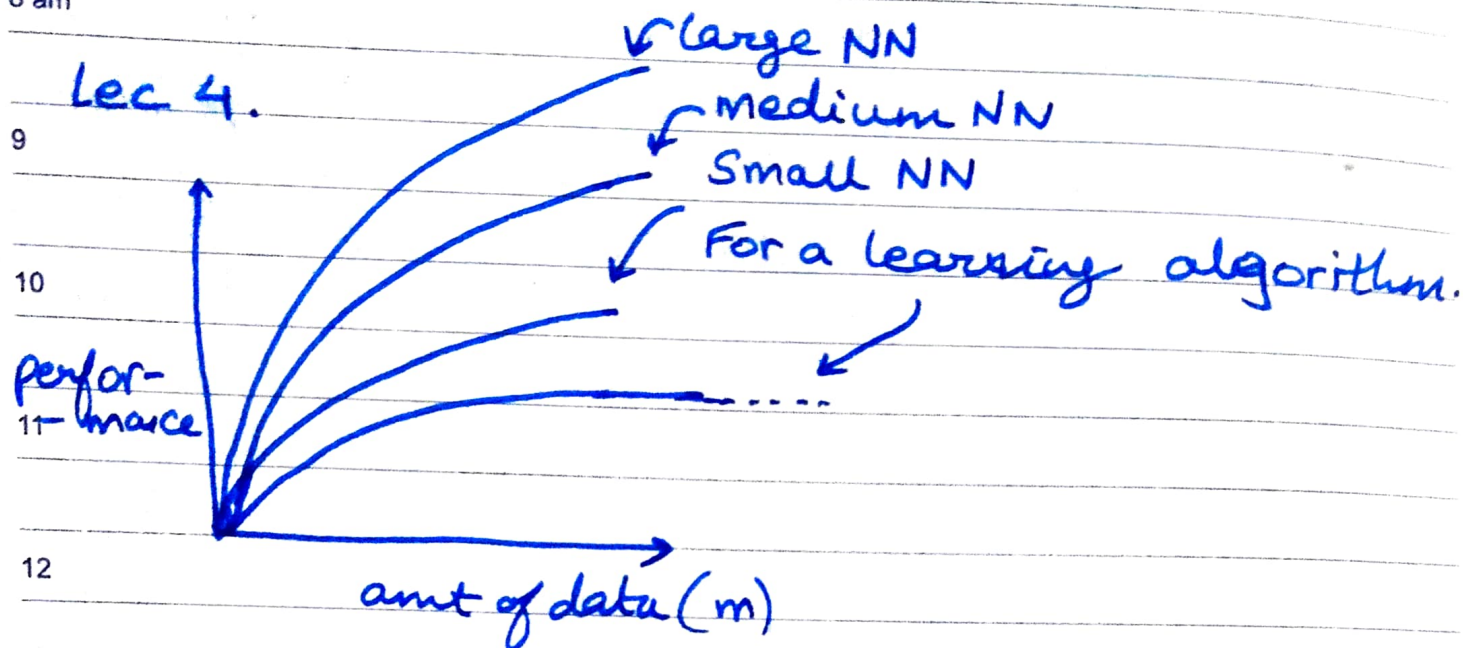
text

17 Sunday

6

8 am

Lec 4.



Today ~~one~~ one of the more reliable ways to get better results is to train a bigger NN, or throw more data at it. However this only works up to a certain point bcz the performance becomes a horizontal asymptote.

Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



This is great for probability o/p's and it was used in earlier neural networks.

- it has a vanishing gradient problem. (learning becomes slow at large values)

8 am

ReLU Function.

$$\text{ReLU}(z) = \underline{1} \max(0, z)$$

9

Deep learning is driven by:

10

- data
- computation

11

Vid 5:

12

Outline:

1 pm

- Introduction
- Basics of Neural Networks
- Code 1 hidden layer of Neural network
- Deep Neural network

2

3

Vid 6: (Resources)

4

5

Vid 7:

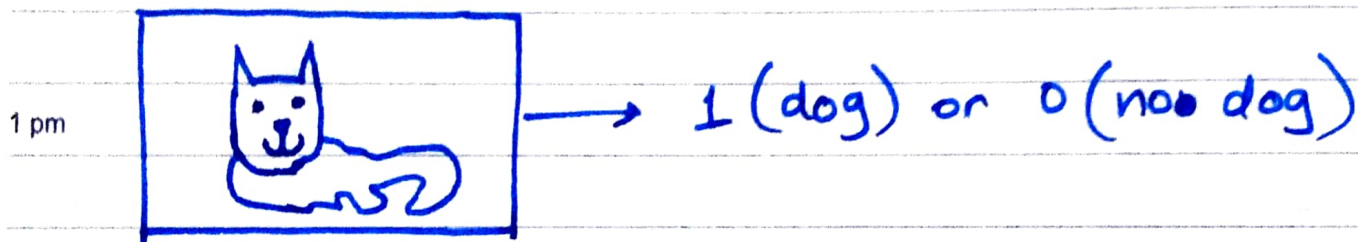
9 Binary Classification.

10 Logistic regression is an algorithm for Binary classification.

11

ex. of Binary Classification.

12



2

PC stores 3 matrices corresponding to R, G, B.

3 the dimensions of these matrices is same as image

4 a feature vector is written as :

5 $X =$ $\begin{bmatrix} r_1 \\ r_2 \\ \vdots \\ b_1 \\ b_2 \\ \vdots \\ g_1 \\ g_2 \\ \vdots \end{bmatrix}$ the number of elements in this will be: $l \times b \times 3$

6

$$n_x = l * b * 3$$

7

8 am A training example. is written as a pair (x, y)

9 where x is an n_x -dimensional feature vector
 $x \in \mathbb{R}^{n_x}$

10 & y is 0 or 1.

11 training set. will have m training examples:

12 $\{ (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), (x^{(3)}, y^{(3)}) \dots (x^{(m)}, y^{(m)}) \}$

1 pm

a matrix X will be defined as:

2

3
$$X = \begin{bmatrix} \uparrow & \uparrow & \uparrow & & \uparrow \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots & x^{(m)} \\ \downarrow & \downarrow & \downarrow & & \downarrow \end{bmatrix}$$

4

5 $X \in \mathbb{R}^{n_x \times m}$

6 $X.\text{shape} = (n_x, m)$

7 $Y = [y^{(1)}, y^{(2)}, \dots, y^{(m)}] \quad Y = \mathbb{R}^{1 \times m}$

$Y.\text{shape} = (1, m)$

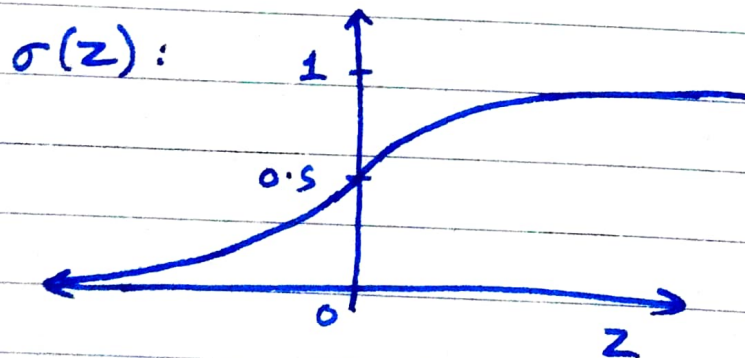
Vid 8: "Logistic Regression"

given an input x (corresponding to image),
we want $\hat{y} = P(y=1 | x)$

$$x \in \mathbb{R}^{n_x}$$

parameters: $w \in \mathbb{R}^{n_x}$, $b \in \mathbb{R}$

o/p: $\hat{y} = \sigma(w^T x + b)$



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

If z is large, $\sigma(z) \approx 1$

z is small, $\sigma(z) \approx 0$

when this is programmed, we have to find $w^T x$ & b

Vid 9: "Cost Function"

9

$$\hat{y} = \sigma(w^T x + b)$$

10

given training set: $\{(x^{(1)}, y^{(1)}), \dots, (x^{(m)}, y^{(m)})\}$
 we want $\hat{y}^{(i)} \approx y^{(i)}$

11

12 Loss (Error Fcn): $\mathcal{L}(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2$

1 pm

↓ measures how good $\hat{y}$ is, where true label is y

2

$$\mathcal{L}(\hat{y}, y) = - (y \log \hat{y} + (1-y) \log(1-\hat{y}))$$

3

If $y = 1$, $\mathcal{L}(\hat{y}, 1) = -\log \hat{y}$

4

If $y = 0$, $\mathcal{L}(\hat{y}, 0) = -\log(1-\hat{y})$

5

It is evident that when $y=1$, $\hat{y}$ is wanted large
 when $y=0$, $\hat{y}$ is wanted small

24 Sunday

6

Cost F^n : $J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$

9

$$= -\frac{1}{m} \sum_{i=1}^m \left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)}) \right]$$

10
11 measures how well the parameters w & b are doing on the training set.

12 Vid 10:

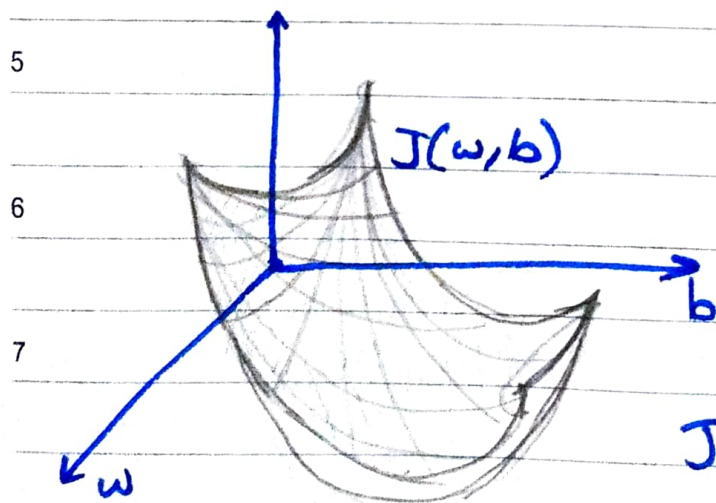
Gradient Descent.

1 pm

we want to find w, b such that $J(w, b)$ reaches its minimum value.

3 $w, b \in \mathbb{R}$.

4 J is some surface above the horizontal axes w, b .



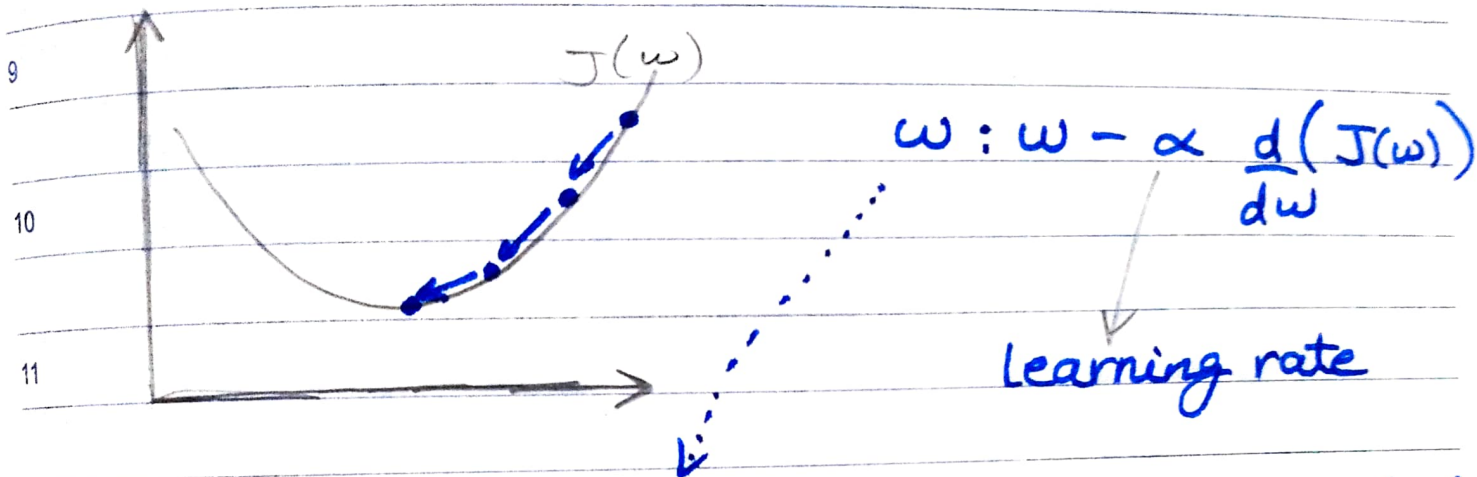
w, b required are values which minimize value.

we use THAT fn as J because of its convex shape.



8 am

lets say there is a fn $J(w)$



12

In each update, a step is taken in that direction, which reduces the cost most rapidly.

1 pm

Vid 11; Vid 12

2

Derivatives

[Basics of derivatives] {nothing new}

3

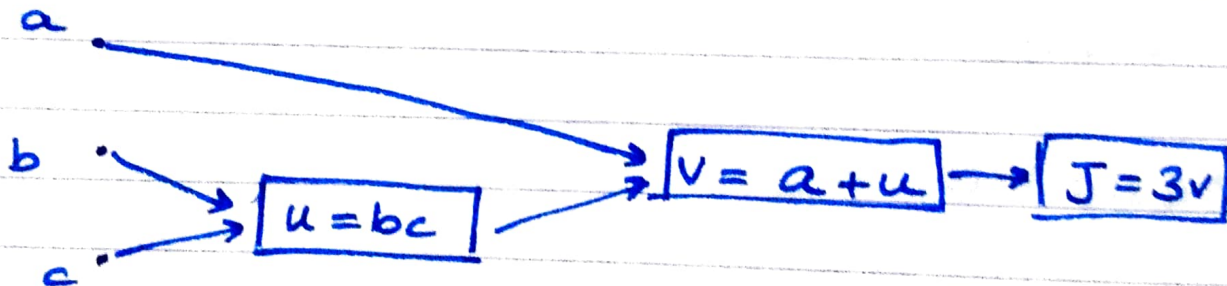
Vid 13

Computation Graph.

eg. $J(a, b, c) = 3(a + \underline{bc})$

• $u = bc$
 $v = a + u$
 $J = 3v$

$$\frac{\frac{J}{v}}{u}$$



This is an example of a computation graph.

Vid 14.

Derivatives using computation graphs.
 [chain rule]

Vid 15: Logistic Regression

$$z = w^T x + b$$

$$\hat{y} = a = \sigma(z)$$

$$\mathcal{L}(a, y) = -[y \log a + (1-y) \log(1-a)]$$

$$\frac{d}{da}(\mathcal{L}(a, y)) = -\frac{d}{da}[y \log a + (1-y) \log(1-a)]$$

$$= -y \cdot \frac{d}{da}(\log a) - (1-y) \frac{d}{da} \log(1-a)$$

$$= \boxed{-\frac{y}{a} + \frac{(1-y)}{1-a}}$$

$$\frac{d(\mathcal{L})}{dz} = \frac{d\mathcal{L}}{da} \cdot \frac{da}{dz}$$

$$= \left(-\frac{y}{a} + \frac{(1-y)}{1-a}\right) a(1-a)$$

$$= -y(1-a) + a(1-y)$$

$$= -y + ay + a - ay$$

$$\boxed{\frac{d(\mathcal{L})}{dz} = a - y}$$

Vid 16: Gradient descent on m examples

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(a^{(i)}, y)$$

$$a^{(i)} = y^{(i)} = \sigma(z^{(i)}) \\ = \sigma(w^T x^{(i)} + b)$$

$$\frac{\partial}{\partial w_1} (J(w, b)) = \frac{1}{m} \sum_{i=1}^m \frac{\partial}{\partial w_1} (\mathcal{L}(a^{(i)}, y^{(i)}))$$

Initializing $J=0$, $dw_1=0$; $dw_2=0$; $db=0$

$$\text{for } i=1 \text{ to } m \{ \\ z^{(i)} = w^T x^{(i)} + b \\ a^{(i)} = \sigma(z^{(i)})$$

$$J += -[y^{(i)} \log a^{(i)} + (1-y^{(i)}) \log(1-a^{(i)})]$$

$$dz^{(i)} = a^{(i)} - y^{(i)}$$

$$\left. \begin{aligned} dw_1 &+= x_1^{(i)} dz^{(i)} \\ dw_2 &+= x_2^{(i)} dz^{(i)} \\ db &+= dz^{(i)} \end{aligned} \right\} \begin{array}{l} \text{for } n=2. \\ \text{for large } n, \text{ will require for-loop} \end{array}$$



$$J /= m$$



$$dw_1 /= m \quad dw_2 /= m \quad db /= m.$$

8 am

$$dw_1 = \frac{\partial J}{\partial w_1}$$

9

$$w_1 : w_1 - \alpha dw_1$$

$$w_2 : w_2 - \alpha dw_2$$

$$b : b - \alpha db$$

11

Vectorization helps us to remove for-loops in our code

12

1 pm

ep Vid 17: Vectorization.

$$z = w^T x + b$$

3

$$w = \begin{bmatrix} \vdots \end{bmatrix} \quad x = \begin{bmatrix} \vdots \end{bmatrix} \quad w \in \mathbb{R}^{n_x} \quad x \in \mathbb{R}^{n_x}$$

4

non vectorized:

5

~~z = 0~~

for i in range(n-x):

6

$$z += w[i] \cdot x[i]$$

$$z += b$$

7

vectorized

$$z = \text{np.dot}(w, x) + b$$

81 Sunday

```
import numpy
```

```
a = np.array([1, 2, 3, 4])
```

```
print(a)
```

o/p: [1, 2, 3, 4]

```
import time
```

```
a = np.random.rand(1000000)
```

```
b = np.random.rand(1000000)
```

```
tic = time.time()
```

```
c = np.dot(a, b)
```

```
toc = time.time()
```

```
print("Vectorized version:" + str(1000*(toc-tic))  
      + "ms")
```

o/p: Vectorized version: 2.98 ms

```
c = 0
```

```
tic = time.time()
```

```
for i in range(1000000):
```

```
    c += a[i] * b[i]
```

```
toc = time.time()
```

```
print("For-loop version:" + str(1000*(toc-tic)) + "ms")
```

o/p: For-loop version: 474.295 ms

we can see the vectorized version takes
for less time than non-vectorized one.

Vid 18: More examples of vectorization.

9

Rule of Thumb: "Whenever possible, avoid for-loop"

10

eg. 1 $U = A \cdot V$

11

NON-VECTORIZED

$$u_i = \sum_j A_{ij} v_j$$

12



1 pm

$u = \text{np.zeros}(10, 1)$

for i in range..

2

for j in range..

3

$u[i] += A[i][j] * v[j]$

VECTORIZED

$u = \text{np.dot}(A, v)$

4

eg. 2

$$v = \begin{bmatrix} v_1 \\ \vdots \\ v_n \end{bmatrix} \rightarrow \begin{bmatrix} e^{v_1} \\ \vdots \\ e^{v_n} \end{bmatrix}$$

5

6

import numpy as np

$u = \text{np.zeros}(n, 1)$

7

for i in range(n):

$u[i] = \text{math.exp}(v[i])$

import numpy as np

$u = \text{np.exp}(v)$



Vid 19: Vectorizing Logistic Regression

20%

$$dz^{(1)} = a^{(1)} - y^{(1)} \dots$$

$$dz^{(2)} = a^{(2)} - y^{(2)} \dots$$

$$dz = [dz^{(1)}, dz^{(2)}, dz^{(3)} \dots dz^{(m)}]$$

$$A = [a^{(1)} \dots a^{(m)}] \quad Y = [y^{(1)} \dots y^{(m)}]$$

$$dz = A - Y = [a^{(1)} - y^{(1)}, a^{(2)} - y^{(2)}, \dots]$$

$$db = \frac{1}{m} \sum_{i=1}^m dz^{(i)}$$

$$= \frac{1}{m} \text{np.sum}(dz)$$

$$dw = \frac{1}{m} X \cdot dz^T$$

$$Z = w^T X + b$$

$$\Rightarrow \text{np.dot}(w^T, X) + b$$

[NO FOR LOOPS!]

$$A = \sigma(Z)$$

$$db = \frac{1}{m} \text{np.sum}(dz)$$

$$dz = A - Y$$

$$dw = \frac{1}{m} \cdot X dz^T$$

$$w = w - \alpha dw$$

$$b = b - \alpha db$$

Vid 21: Broadcasting:

9

10 calories from carb, protein, fats in 100g of diff foods:

	apple	eggs	Potatoes
carb	56	4.4	68
protein	1.2	52.0	8
fat	1.8	99.0	0.9

12

1 pm

$$\text{cal} = A \cdot \text{sum}(\text{axis}=0)$$

2

$$\text{percentage} = 100 * A / (\text{cal} \cdot \text{reshape}(1,3))$$

3

eg. of Broadcasting

4

$$\begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} + 100 = \begin{bmatrix} 101 \\ 102 \\ 103 \\ 104 \end{bmatrix}$$

5

6

eg.

7

$$\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ \vdots & \vdots & \vdots \\ x_{m1} & x_{m2} & x_{m3} \end{bmatrix} + \begin{bmatrix} 100 & 200 & 300 \end{bmatrix} = \begin{bmatrix} 100+x_{11}, 200+x_{12} \\ 100+x_{21}, 200+x_{22} \\ \vdots \\ 100+x_{m1}, \dots \end{bmatrix}$$



Vid 22.

Don't use rank 1 arrays.

$\text{np.random.randn}(5) \times$
 $\text{np.random.randn}(5,1) \checkmark$

Vid 24: Explanation of Cost Function.

$$\hat{y} = \sigma(w^T x + b) \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Interpret $\hat{y} = P(y=1 | x)$

$$\begin{cases} \text{If } y=1 & : & P(y|x) = \hat{y} \\ y=0 & : & P(y|x) = 1 - \hat{y} \end{cases}$$

$$\rightarrow P(y|x) = \hat{y}^y (1 - \hat{y})^{1-y} \quad **$$

$$\begin{aligned} \log[P(y|x)] &= y \log \hat{y} + (1-y) \log(1 - \hat{y}) \\ &= -\mathcal{L}(y, \hat{y}) \end{aligned}$$

9

$$P(\text{labels in training set}) = \prod_{i=1}^m P(y_i | x_i)$$

10

$$\log P(\text{"}) = \sum_{i=1}^m \log P(y_i | x_i)$$

$$= - \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

11

$$= - \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

12

to find parameters that maximize $P(\text{" " "})$

1 pm

2

$$\text{Cost : } J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

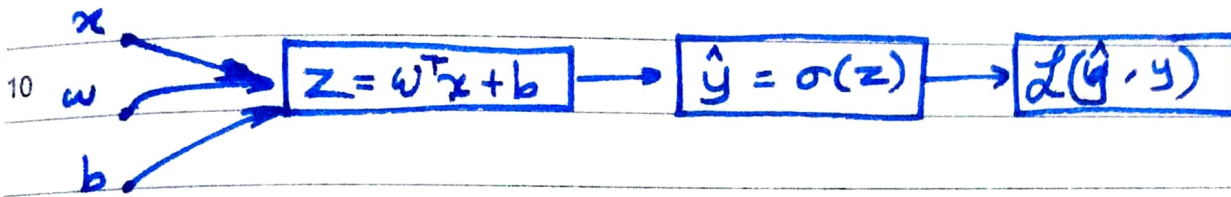
3

by minimizing $J(w, b)$ we are maximizing likelihood estimation of logistic Regression model.

4

Vid 25 : Neural Network Overview.

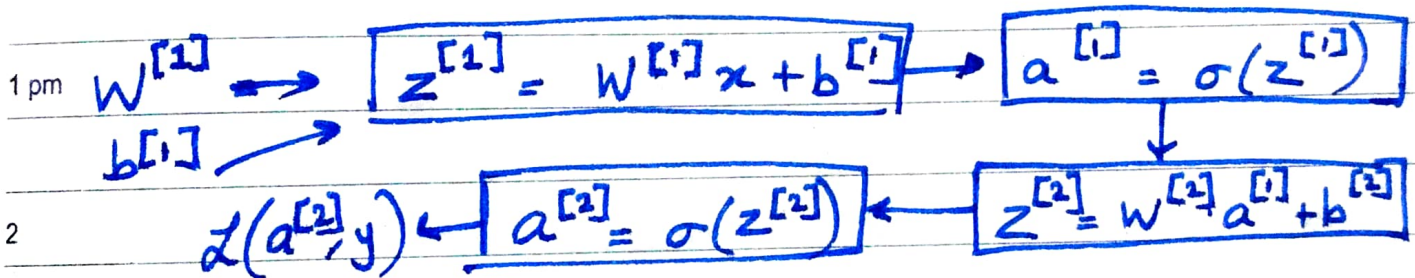
9



11

$[] \rightarrow$ layers in NN $() \rightarrow$ individual training set

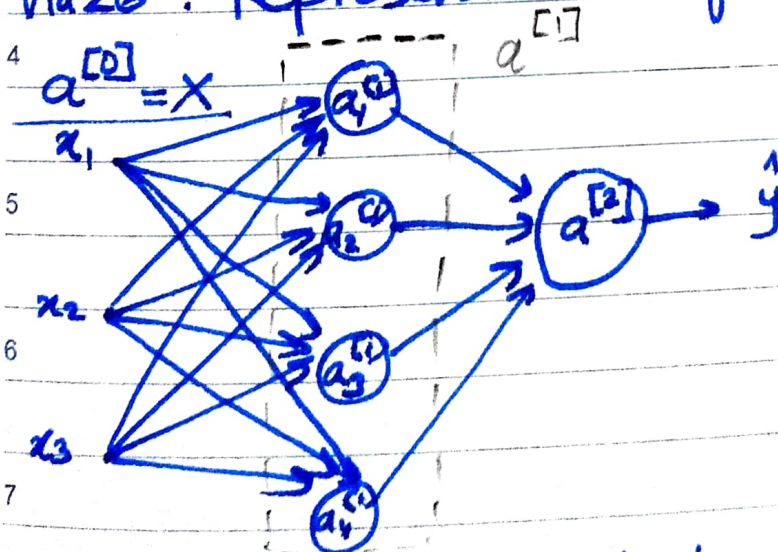
12



3

Vid 26 : Representation of NN

4



$$a^{[1]} = \begin{bmatrix} a_1^{[1]} \\ a_2^{[1]} \\ \vdots \\ a_n^{[1]} \end{bmatrix} \quad \text{07 Sunday}$$

Input layer hidden layer output layer

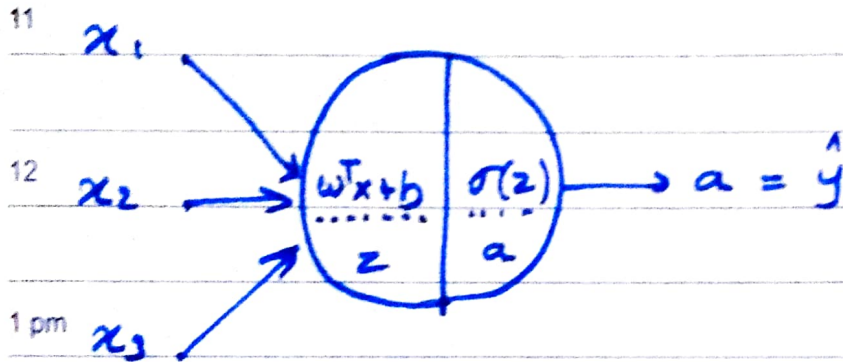


2 layer NN

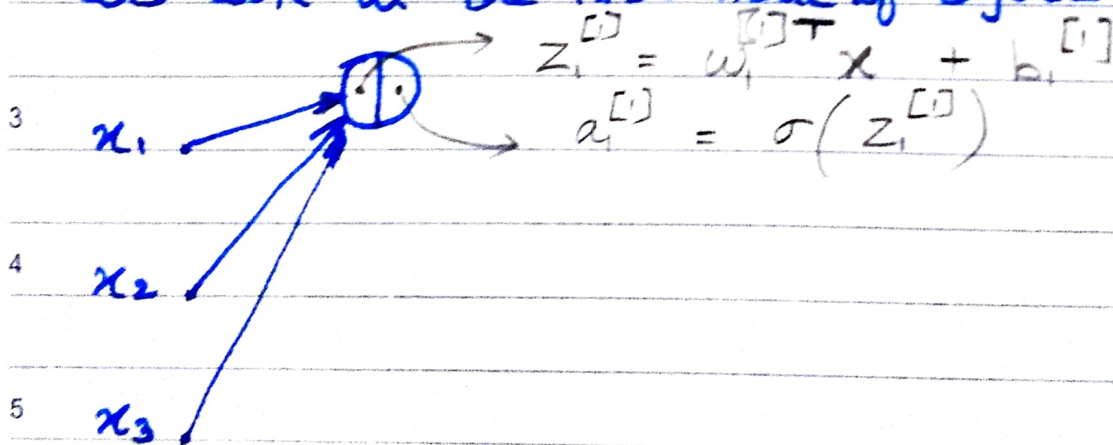
$w^{[1]}, b^{[1]}$ will be parameters associated with $a^{[1]}$

Vid 27.

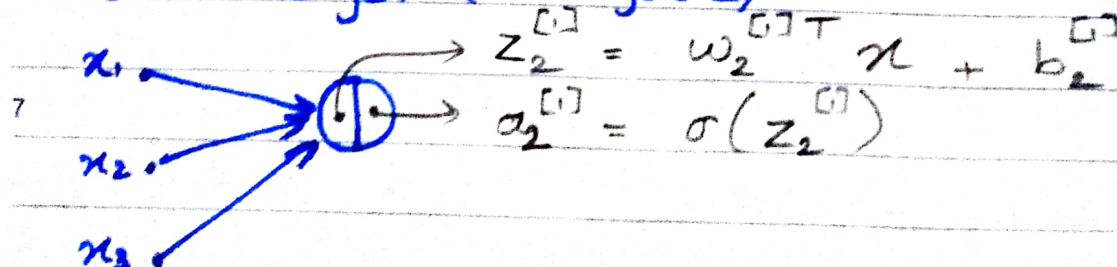
Computing Neural Network



Let's look at the first node of layer 1



Second node (of layer 1)



8 am

For four nodes, we get:

$$\begin{aligned} \underline{Z}_1^{[1]} &= \underline{w}_1^{[1]T} x + \underline{b}_1^{[1]} ; \quad \underline{Z}_2^{[1]} = \underline{w}_2^{[1]T} x + \underline{b}_2^{[1]} \\ \underline{Z}_3^{[1]} &= \underline{w}_3^{[1]T} x + \underline{b}_3^{[1]} ; \quad \underline{Z}_4^{[1]} = \underline{w}_4^{[1]T} x + \underline{b}_4^{[1]} \end{aligned}$$

10

It seems that we can apply for loop for multiple nodes, but we can avoid this using Vectorization.

11

$$\begin{bmatrix} \underline{w}_1^{[1]T} \\ \underline{w}_2^{[1]T} \\ \underline{w}_3^{[1]T} \\ \underline{w}_4^{[1]T} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} \underline{b}_1^{[1]} \\ \underline{b}_2^{[1]} \\ \underline{b}_3^{[1]} \\ \underline{b}_4^{[1]} \end{bmatrix} = \begin{bmatrix} \underline{Z}_1^{[1]} \\ \underline{Z}_2^{[1]} \\ \underline{Z}_3^{[1]} \\ \underline{Z}_4^{[1]} \end{bmatrix} = \underline{Z}^{[1]}$$

1 pm

4x3 matrix

FIRST NODE

2

3

4

$$\underline{a}^{[1]} = \begin{bmatrix} a_1^{[1]} \\ \vdots \\ a_n^{[1]} \end{bmatrix} = \sigma(\underline{Z}^{[1]})$$

Vid 28: Vectorizing Across multiple examples.

$$X \longrightarrow a^{[2]} = \hat{y}$$

for $i = 1$ to m .

$$z^{[1](i)} = W^{[1]} x^{(i)} + b^{[1]}$$

$$a^{[1](i)} = \sigma(z^{[1](i)})$$

$$z^{[2](i)} = W^{[2]} a^{[1](i)} + b^{[2]}$$

$$a^{[2](i)} = \sigma(z^{[2](i)})$$

we want to vectorize to avoid for-loops.

$$X = \begin{bmatrix} \vdots & \vdots & \vdots \\ x^{(1)} & x^{(2)} & \dots & x^{(m)} \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix}$$

(n_x, m)

To vectorized :

$$Z^{[1]} = W^{[1]} X + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$$

$$Z^{[1]} = \begin{bmatrix} \vdots & \vdots & & \vdots \\ z^{[1]}(1) & z^{[1]}(2) & \dots & z^{[1]}(m) \\ \vdots & \vdots & & \vdots \end{bmatrix}$$

$$A^{[1]} = \begin{bmatrix} \circ & \vdots & \vdots & \vdots \\ a^{[1]}(1) & a^{[1]}(2) & \dots & a^{[1]}(m) \\ \vdots & \vdots & & \vdots \end{bmatrix}$$

First hidden unit → 2nd hidden unit

* training examples

So,

$$A = \begin{bmatrix} \\ \\ \\ \end{bmatrix} \quad \downarrow \quad \# \text{ hidden units.}$$



Vid 29: Explanation for vectorized implementation

9

$$\begin{aligned} z^{[1]}(1) &= W^{[1]} x^{(1)} + b^{[1]}, & // \text{ training example 1} \\ z^{[1]}(2) &= W^{[1]} x^{(2)} + b^{[1]}, & // \text{ training example 2} \end{aligned}$$

11

$$W^{[1]} = \begin{bmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ - & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \end{bmatrix} \quad W^{[1]} x^{(1)} = \begin{bmatrix} \cdot \\ \cdot \\ \cdot \\ \cdot \end{bmatrix}, \quad W^{[1]} x^{(2)} = \begin{bmatrix} \cdot \\ \cdot \\ \cdot \\ \cdot \end{bmatrix}$$

12

1 pm

$$X = \begin{bmatrix} \vdots & \vdots & \vdots & \dots \\ x^{(1)} & x^{(2)} & x^{(3)} & \dots \\ \vdots & \vdots & \vdots & \dots \end{bmatrix}$$

2

3

stack horizontally for more m.

4

5 when this matrix X is multiplied to W, we get

6

7

$$W^{[1]} X = \begin{bmatrix} \vdots & \vdots & \vdots & \dots \end{bmatrix}$$

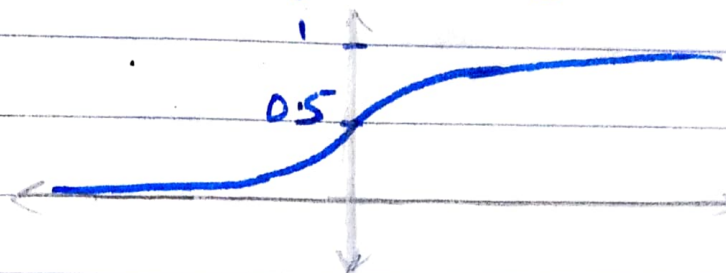


when $b^{[1]}$ is added, we get $z^{[1]}$

Vid 30: ACTIVATION FUNCTION.

- So far, we have been using the sigmoid F^n .

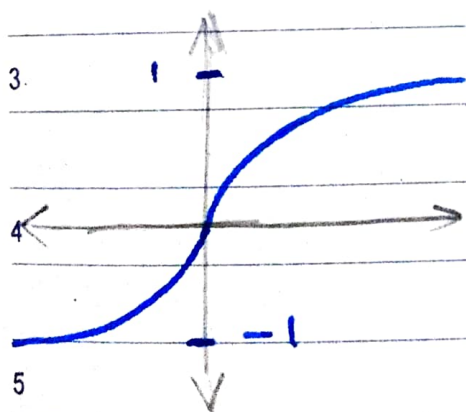
$$\sigma(z) = \frac{1}{1+e^{-z}}$$



$\sigma(z)$ is an example of an activation function.

we can replace $\sigma(z)$ with any general $f^n g(z)$

another example is the $\tanh(z)$ function.



$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

This can be used since, at times

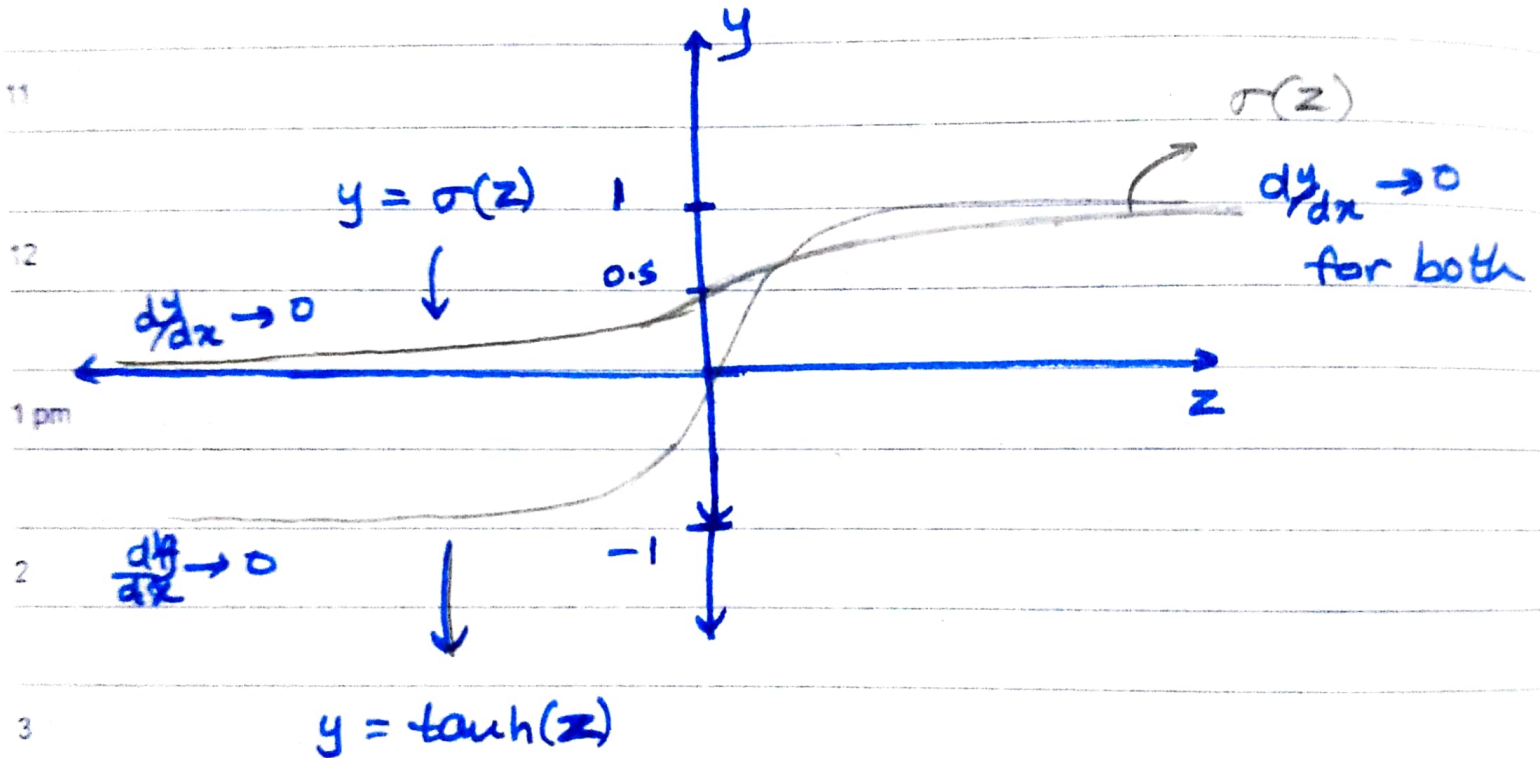
we want the mean to be centered at 0. ^{14 Sunday}

since Range = $(-1, 1)$

The $\tanh$ fn is almost always better than $\sigma(z)$
EXCEPT at the o/p layer. $\therefore y \in \{0, 1\}$

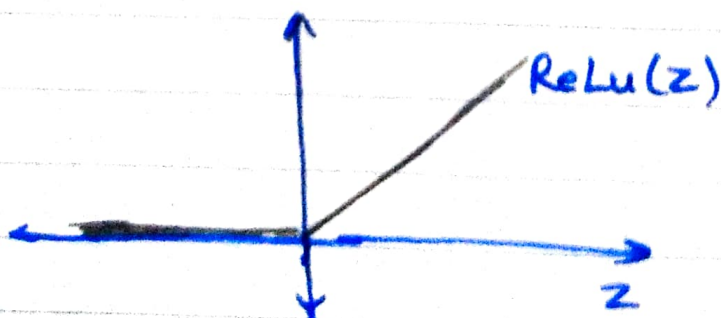
• One disadvantage of both these f^{ns} is that

if z is very large or very small, then the gradient / slope becomes very small.



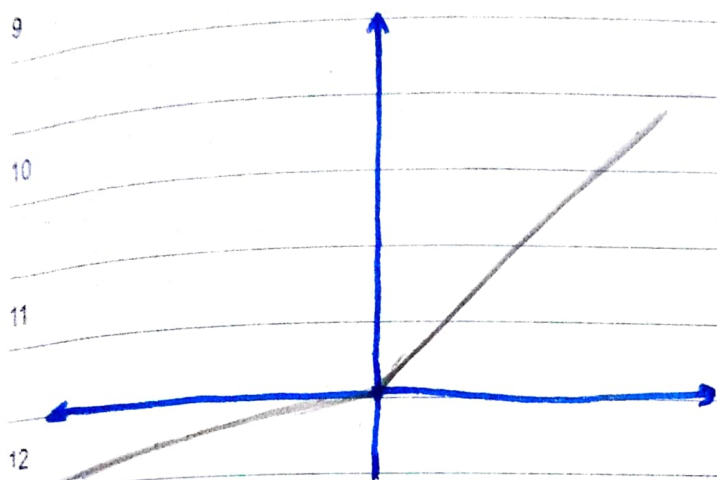
To solve these issues, a popular function in M.L. used is Rectified Linear Union (ReLU)

$$\text{ReLU}(z) = \max(0, z)$$



not differentiable
at $z = 0$

8 am
"leaky ReLU": $a = \max(0.01z, z)$



Vid 31. Why NON-LINEAR activation fⁿs?

2
3
If we use $a^{[1]} = z^{[1]}$ instead of $a^{[1]} = g(z^{[1]})$

4
5
then the whole network becomes linear, which eliminates the point of having multiple layers.

6
7
$$z^{[1]} = W^{[1]}x + b^{[1]}$$

8
$$a^{[1]} = z^{[1]}$$

9
$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

10
$$= W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]}$$

11
$$= (W^{[2]}W^{[1]})x + (W^{[2]}b^{[1]} + b^{[2]})$$

12
$$= W'x + b'$$



No matter how many layers are added, it is equivalent to a single layer model.

8 am

GPT's use the function :

9 Gaussian Error Linear Unit (GELU)

$$10 \quad \text{GELU}(z) = z \cdot \frac{1}{2\pi} \int_{-\infty}^z e^{-t^2/2} dt$$

11

12 Vid 32 : Derivatives of Activation Functions.

$$\begin{aligned}
 1 \text{ pm} \quad \frac{d}{dz} \sigma(z) &= \frac{d}{dz} \left[\frac{1}{1+e^{-z}} \right] \\
 2 \quad &= \frac{-1}{(1+e^{-z})^2} \cdot (-e^{-z}) \\
 3 \quad &= \frac{e^{-z}}{(1+e^{-z})^2} \\
 4 \quad &= \frac{1}{1+e^{-z}} \cdot \left[1 - \frac{1}{1+e^{-z}} \right] \\
 5 \quad &= \sigma(z) \cdot [1 - \sigma(z)] \\
 &= a(1-a)
 \end{aligned}$$

6 when $z \uparrow \uparrow$, $\frac{d}{dz}(\sigma(z)) \rightarrow 0$
 or $z \downarrow \downarrow$

7

8 am

$$\frac{d}{dz}(\tanh) = \frac{d}{dz} \left[\frac{e^z - e^{-z}}{e^z + e^{-z}} \right]$$

9

$$= 1 - (\tanh)^2$$

10

$$= 1 - (\tanh)^2$$

11

$$\text{if } z \uparrow\uparrow, \quad \frac{d}{dz}(\tanh(z)) \rightarrow 0$$

or $z \downarrow\downarrow$

12

1 pm

$$\frac{d}{dz}(\text{ReLU}(z)) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z > 0 \\ \text{undefined} & \text{if } z = 0 \end{cases}$$

(doesn't matter)

2

Vid 33. Gradient Descent for NN's.

3

parameters : $w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}$ (for single layer)

4

$$\text{cost fn: } J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}_i, y_i)$$

5

It is better to assign random values to parameters rather than zeroes. # Vid 35

6

$$dw^{[1]} = \frac{dJ}{dw^{[1]}}$$

$$db^{[1]} = \frac{dJ}{db^{[1]}}$$

$$w^{[1]} := w^{[1]} - \alpha dw^{[1]}$$

$$b^{[1]} := b^{[1]} - \alpha db^{[1]}$$

★ FORMULAS FOR FORWARD PROP.

$$z^{[1]} = w^{[1]}x + b^{[1]}$$

$$A^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = w^{[2]} \cdot A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(z^{[2]}) \bullet =$$

★ FORMULAE FOR BACKPROP.

$$dz^{[1]} = A^{[2]} - y$$

$$dw^{[2]} = \frac{1}{m} dz^{[2]} A^{[1]T}$$

$$db^{[2]} = \frac{1}{m} \cdot \text{np.sum}(dz^{[2]}, \text{axis}=1, \text{keepdims}=\text{true})$$

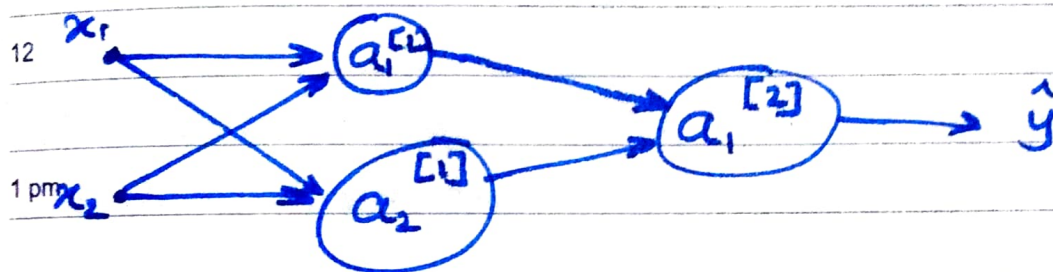


$$dw^{[1]} = \frac{1}{m} dz^{[1]} x^T$$

8 am $db^{[1]} = \frac{1}{m} \text{np.sum}(dz^{[1]}, \text{axis} = 1, \text{keepdims} = \text{true})$

Vid 35. Random Initialization.

First let's see what happens if you initialize all weights to zero.



$$W^{[1]} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \quad b^{[1]} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$a_1^{[1]} = a_2^{[1]}$$

when backpropagation is finished,

$$dz_1^{[1]} = dz_2^{[1]}$$

$$W^{[2]} = \begin{bmatrix} 0 & 0 \end{bmatrix}$$

the 2 hidden units, compute the same thing after many iterations.

$\therefore$ There will be no point in having multiple hidden units.

21 Sunday

∴ parameters must be initialized randomly.

$$W^{[1]} = \text{np.random.randn}[(2,2)] * 0.01$$

$$b^{[1]} = \text{np.zeros}[(2,1)]$$

b does not have the symmetry problem, ∴ its okay to initialize it as 0.

this is chosen small

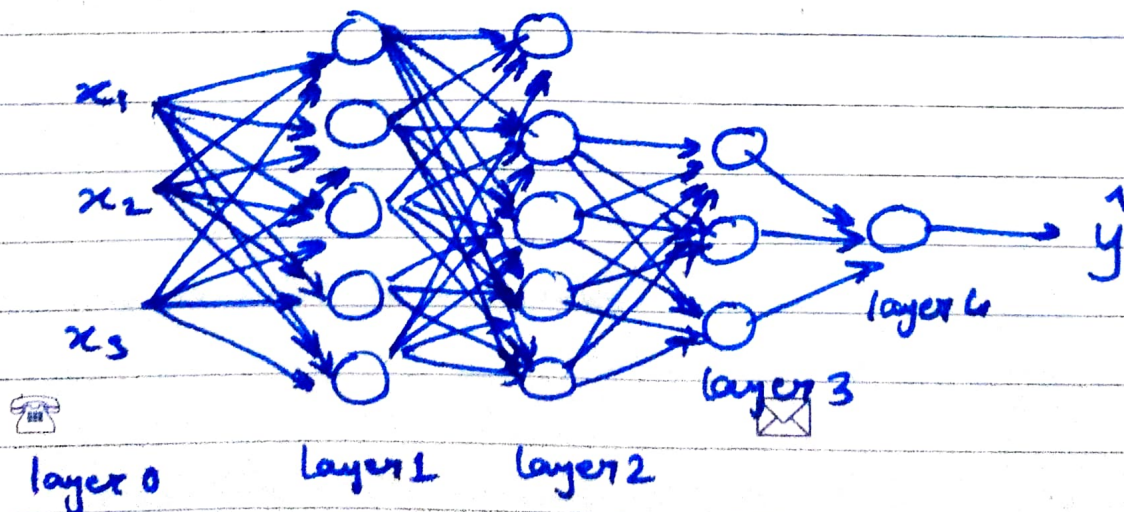
because, if $W \uparrow \uparrow$, then you're more likely to end up at flatter parts of $g(z)$.

Vid 36. Deep L-Layer Neural Network.

"deep" means more hidden layers.

$l = \# \text{ layers}$

$n^{[l]} = \# \text{ units in } l^{\text{th}} \text{ layer.}$



$$n^{[1]} = 5$$

$$n^{[2]} = 5$$

$$n^{[3]} = 3$$

$$n^{[4]} = 1 = n^{[1]}$$

$a^{[l]}$ = activations in layer l .

$$a^{[1]} = g^{[1]}(z^{[1]})$$

Vid 37. FORWARD PROP IN DEEP NETWORK.

$$X: \quad z^{[1]} = W^{[1]}x + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$\vdots$

$$z^{[4]} = W^{[4]}a^{[3]} + b^{[4]}$$

$$a^{[4]} = g^{[4]}(z^{[4]})$$

$$z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Vectorized version:

9

$$Z^{[1]} = W^{[1]} A^{[0]} + b^{[1]}$$

10

$$A^{[1]} = g^{[1]}(Z^{[1]})$$

11

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$$

12

$$A^{[2]} = g^{[2]}(Z^{[2]})$$

1 pm

$$\hat{Y} = g(Z^{[4]}) = A^{[4]}$$

for loop.

$l = 1, 2, 3, 4.$

2

Vid 38. Getting Matrix Dimensions right.

3

$l = 5.$

4

$$Z^{[1]} = W^{[1]} x + b^{[1]}$$

5

$$\begin{pmatrix} 3 \\ \vdots \\ 1 \end{pmatrix} \downarrow$$

$$(n^{[1]} \times 1)$$

6

$$[n^{[1]} \times n^{[0]}] [n^{[0]}, 1]$$

7

dimensions of $W^{[2]} = n^{[2]} \times n^{[1]}$

" of $W^{[3]} = n^{[3]} \times n^{[2]}$

$W^{[4]} = n^{[4]} \times n^{[3]}$

$$\therefore \dim(W^{[L]}) = (n^{[L]} \times n^{[L-1]})$$

$$\dim(b^{[L]}) = \dim(z^{[L]}) = (n^{[L]} \times 1)$$

$$\dim[dw^{[L]}] = (n^{[L]} \times n^{[L-1]})$$

$$\dim[db^{[L]}] = (n^{[L]} \times 1)$$

in Vectorized form:

$$Z^{[1]} : [z^{1}, z^{[1](2)} \dots z^{[1](m)}]$$

$$\therefore \dim(Z^{[1]}) = n^{[1]} \times m.$$

$$Z^{[1]} = W^{[1]} X + b^{[1]}$$

$$(n^{[1]}, m)$$

$$(n^{[1]}, n^{[0]})$$

$$(n^{[0]}, m)$$

originally: $n^{[1]}, 1$

$$(n^{[1]}, m)$$

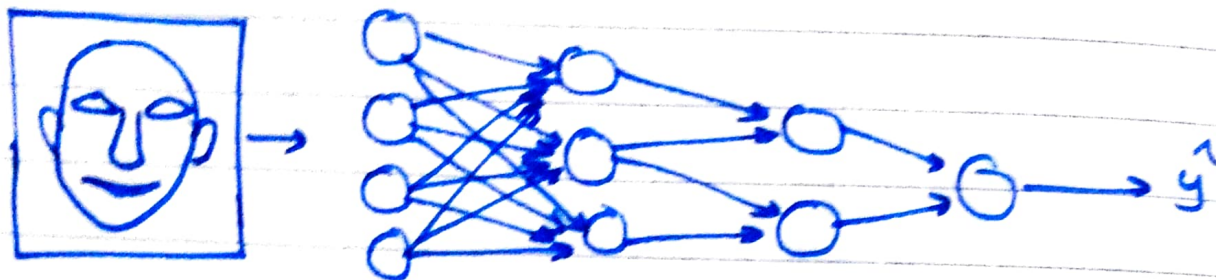
(due to broadcasting)

Vid 39. Deep Representations.

9

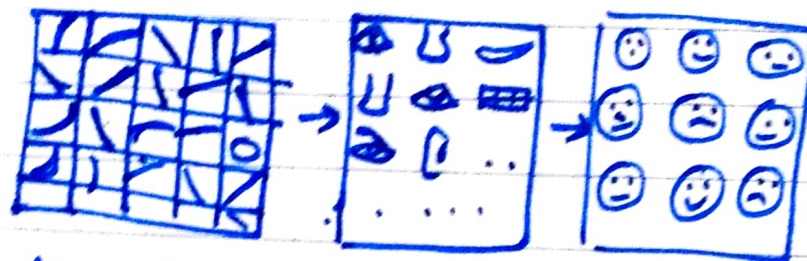
10

11



12

1 pm



2

3

the earlier layer detect simple features, and as the NN goes deeper, more complex features are detected.

Vid 40.

4

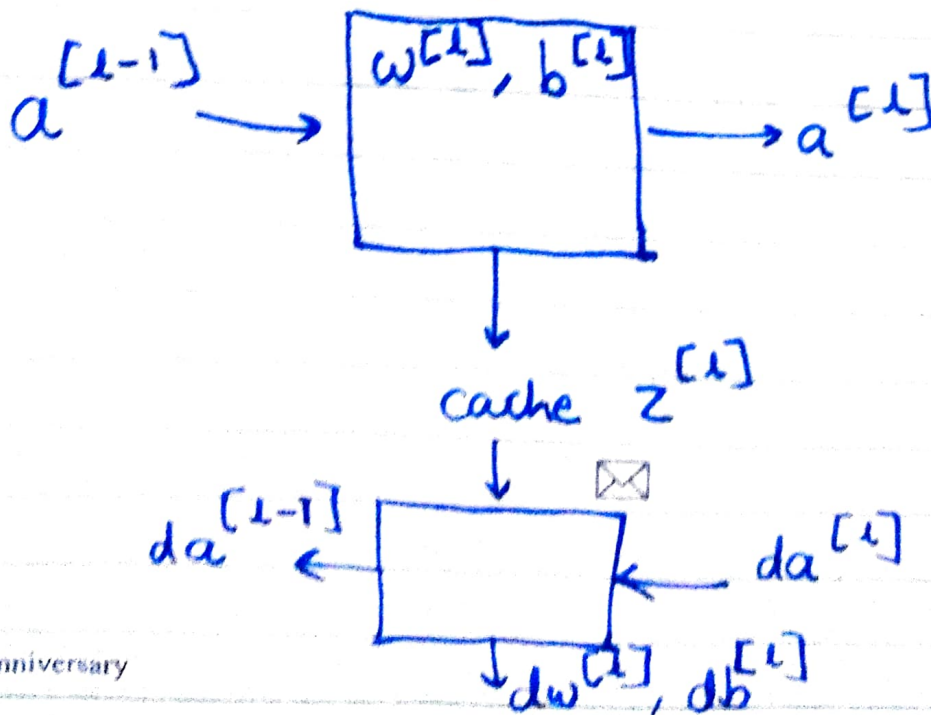
Basic structure of DNN.

5

layer 1

6

7



8 am

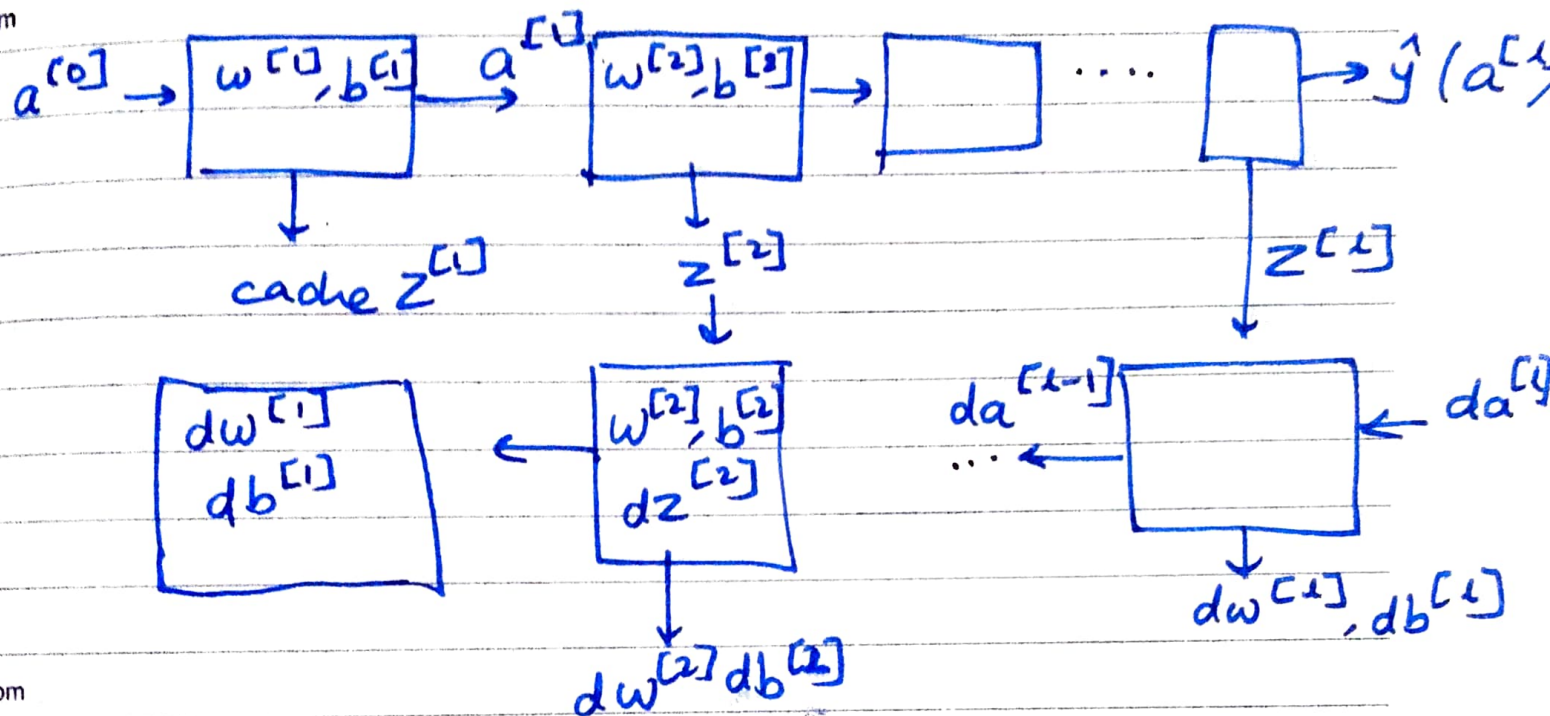
9

10

11

12

1 pm



2

Vid 40.

FORWARD PROP & BACKPROP.

3

4

5

6

7

input $a^{[l-1]}$
 Output $a^{[l]}$, cache $(z^{[l]})$ } FORWARD PROP.

input $da^{[l]}$
 Output $da^{[l-1]}$, $dw^{[l]}$, $db^{[l]}$

28 Sunday

Vectorized version :

$$dZ^{[L]} = dA^{[L]} * g^{[L]}(Z^{[L]})$$

$$dW^{[L]} = \frac{1}{m} dZ^{[L]} \cdot A^{[L-1]T}$$

$$db^{[L]} = \frac{1}{m} \text{np.sum}(dZ^{[L]}, \text{axis} = 1, \text{keepdims} = \text{true})$$

$$dA^{[L-1]} = W^{[L]T} \cdot dZ^{[L]}$$

Vid 42.

Parameters & Hyperparameters.

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, \dots$

Hyperparameters: $\rightarrow \alpha$ (learning rate)

$\rightarrow$ # iterations.

$\rightarrow$ # hidden layers

$\rightarrow$ # hidden units

$\rightarrow$ choice of Activation fn. $g(z)$
such as $\sigma(z)$, $\text{ReLU}(z)$, $\tanh(z)$... etc.

(These control W, b)